TRƯỜNG ĐẠI HỌC BÁCH KHOA – ĐẠI HỌC ĐÀ NẴNG
KHOA ĐIỆN TỬ VÀ TRÍ TUỆ NHÂN TẠO

ĐỀ TÀI

Lập trình Bất đồng bộ với Future & Stream trong Flutter

Sinh viên thực hiện: **Phan Văn Trà**
Trần Đăng Duy

Giảng viên hướng dẫn: **TS. Nguyễn Duy Nhật Viễn**

Mục lục

1. Cơ sở lý thuyết
- 1.1 Cơ chế bất đồng bộ: Future & async/await
- 1.2 Luồng dữ liệu liên tục: Stream
- 1.3 Cơ chế dọn dẹp bộ nhớ

2. Giới thiệu project FocusFlow (Pomodoro)
- 2.1 Tổng quan project
- 2.2 Áp dụng Future & Stream và luồng hoạt động
- 2.3 Demo sản phẩm

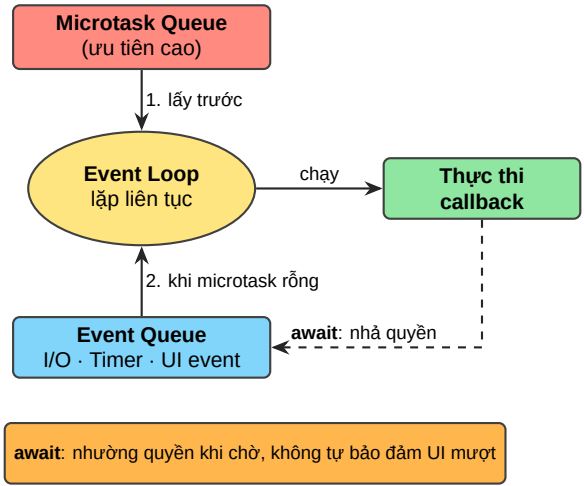
3. Kết luận

Phân công công việc

TT	HỌ VÀ TÊN	NỘI DUNG	TỶ LỆ ĐÓNG GÓP
1	Phan Văn Trà	**models/ và services/**: xây dựng model trạng thái, bản ghi phiên; xử lý đếm giờ, chuyển phiên, phát Stream và lưu/đọc lịch sử.	**50%**
2	Trần Đăng Duy	**screens/ và widgets/**: xây dựng màn hình đồng hồ, lịch sử, chỉnh thời lượng; hiển thị Future/Stream và các nút, vòng tiến trình.	**50%**

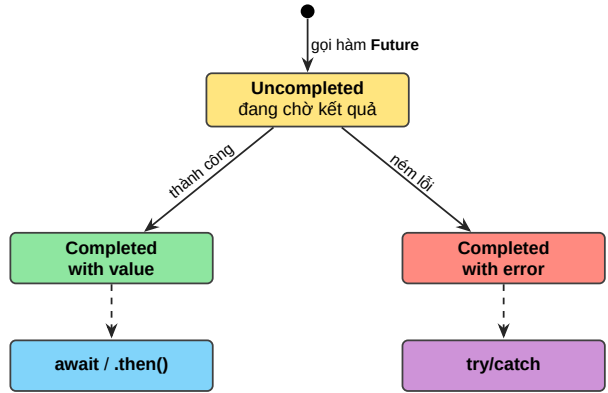
1.1 Vì sao cần lập trình bất đồng bộ?

- Dart chạy trên **một luồng (single-thread)** cho mỗi **isolate**.
- Tác vụ chờ: **mạng, đọc/ghi file, timer, cơ sở dữ liệu** … nếu chặn luồng chính ⇒ **UI bị đứng (jank)**.
- **Event Loop** lần lượt lấy sự kiện từ hai hàng đợi:
  1. **Microtask Queue** (ưu tiên cao)
  2. **Event Queue** (I/O, Timer, UI event)
- **Future** và **Stream** là hai công cụ cốt lõi của Dart để làm việc với Event Loop.



1.1 Future: đại diện cho một kết quả “sẽ có trong tương lai”

- **`Future<T>`** biểu diễn **một giá trị duy nhất** (hoặc một **lỗi**) khi thao tác hoàn thành.
- Hai trạng thái chính: **Uncompleted** và **Completed** (có giá trị hoặc lỗi).
- Nhận kết quả bằng:
  - **`.then()`** để nối thao tác lưu
  - **`async/await`** + **`try/catch`**
- Trong project: **`loadSessions()`** và **`saveSession()`** trả Future.



1.1 Async/await trong thao tác lưu phiên

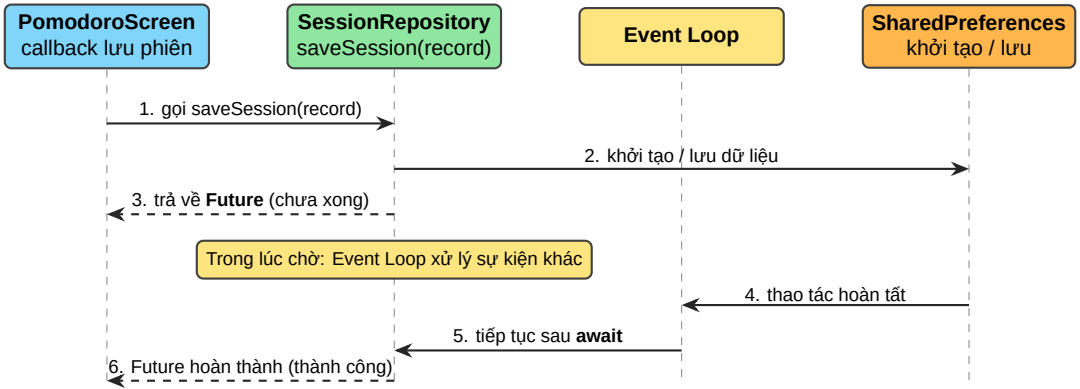
```
Future<void> saveSession(SessionRecord session) async {
  final sessions = await loadSessions();
  sessions.add(session);
  await saveSessions(sessions);
}
```

Future<void>: không trả dữ liệu; bên gọi vẫn có thể chờ hoàn thành và bắt lỗi.

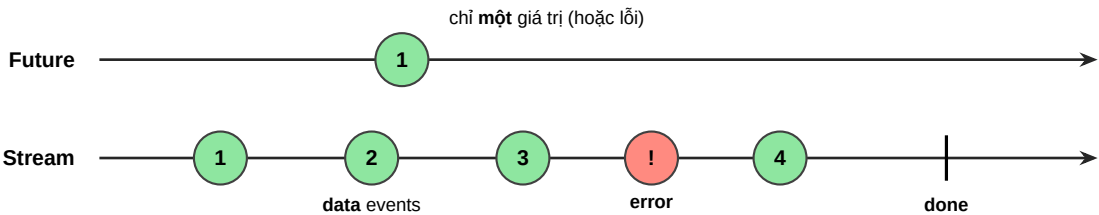
Trong lúc chờ Future, các sự kiện khác có cơ hội được xử lý. Phần thêm vào danh sách vẫn chạy đồng bộ.

1. **`async`**: hàm trả về Future để bên gọi theo dõi việc lưu.
2. **`await loadSessions()`**: đợi lấy danh sách cũ rồi mới chạy dòng tiếp theo.
3. **`sessions.add(session)`**: thêm phiên vừa hoàn thành vào danh sách trong bộ nhớ.
4. **`await saveSessions(sessions)`**: lưu lại toàn bộ danh sách đã cập nhật.

1.1 Sơ đồ tuần tự: luồng chạy của async/await



1.2 Stream: chuỗi sự kiện bất đồng bộ theo thời gian



Tiêu chí	Future	Stream
Số giá trị	**Một** giá trị (hoặc lỗi)	Có thể phát nhiều sự kiện theo thời gian
Kết thúc	Tự kết thúc sau khi có kết quả	Khi nguồn phát gửi **done**
Dùng trong project	`await`, `then`	`StreamBuilder`
Widget Flutter	`FutureBuilder`	`StreamBuilder`

1.2 StreamController trong project

```
final StreamController<PomodoroState> _stateController
    =
    StreamController<PomodoroState>.broadcast();

Stream<PomodoroState> get stream => _stateController.
    stream;
```

```
void _emit(PomodoroState nextState) {
  if (_disposed) return;
  _state = nextState;
  _stateController.add(nextState);
}
```

- **`StreamController`**: nguồn phát dữ liệu kiểu PomodoroState cho đồng hồ.
- **`broadcast()`**: cho phép nhiều bên cùng lắng nghe; không tự phát lại dữ liệu cũ.
- Getter **`stream`**: đưa luồng dữ liệu cho UI; quyền phát vẫn nằm trong bộ đếm.
- **`_disposed`**: dừng hàm ngay nếu bộ đếm đã được giải phóng.
- **`_state = nextState`**: lưu trạng thái hiện tại; **`add(nextState)`** gửi trạng thái đó tới bên lắng nghe.

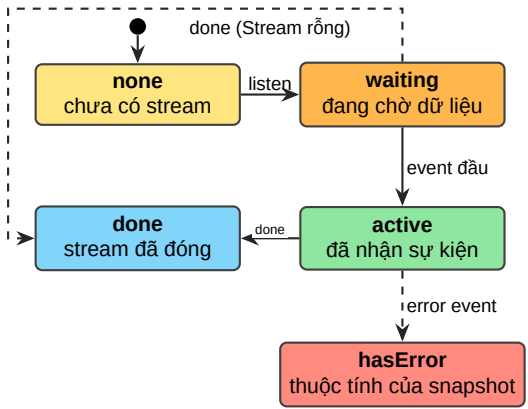
1.2 StreamBuilder: cập nhật giao diện theo Stream

```
body: StreamBuilder<PomodoroState>(
  stream: _timer.stream,
  initialData: _timer.state,
  builder: (context, snapshot) {
    final state = snapshot.data ?? _timer.state;
```

```
Center(
  child: TimerDisplay(state: state, accent:
     accent),
),
```

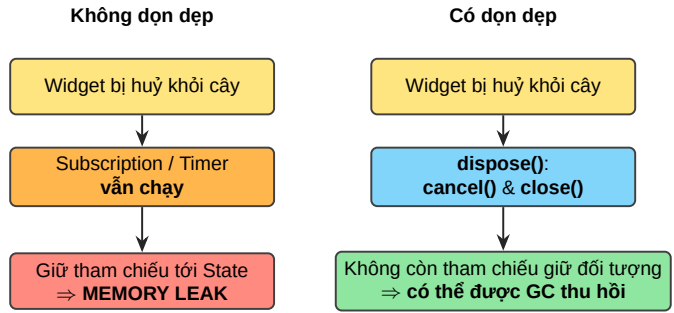
- **`stream`**: theo dõi trạng thái do bộ đếm phát; widget tự đăng ký và hủy lắng nghe.
- **`initialData`**: dùng trạng thái hiện tại ngay trước khi nhận sự kiện đầu tiên.
- **`snapshot.data ?? _timer.state`**: lấy dữ liệu mới, dùng trạng thái hiện tại nếu chưa có dữ liệu.
- **`TimerDisplay`**: nhận state để vẽ thời gian và tiến trình. Sự kiện mới lên lịch cập nhật UI theo nhịp khung hình.

ConnectionState của AsyncSnapshot



1.3 Memory leak khi quên cancel / dispose

- **Timer/subscription** còn chạy có thể giữ tham chiếu tới **State**.
- State còn được tham chiếu có thể **chưa được GC thu hồi**.
- Hậu quả: **memory leak**, tốn CPU/pin, lỗi **`setState() called after dispose()`**.
- Giải pháp: **hủy tài nguyên do mình quản lý** trong **`dispose()`**.



Dọn tài nguyên: hủy Timer, đóng controller do mình quản lý.
mounted bảo vệ cập nhật UI sau await, không thay việc hủy tài nguyên.

1.3 Giải phóng tài nguyên trong project

PomodoroTimer.dispose()

```
void dispose() {
  if (_disposed) return;
  _disposed = true;
  _stopTicker();
  _sessionStartedAt = null;
  unawaited(_stateController.close());
}
```

- **`if (_disposed) return`**: ngăn dọn dẹp lặp lại; gán **`true`** để chặn phát state mới.
- **`_stopTicker()`**: hủy Timer và xóa mốc kết thúc. Dòng tiếp theo xóa thời điểm bắt đầu phiên.
- **`close()`**: đóng controller. **`unawaited()`** thể hiện không chờ Future hoàn tất trong hàm này.

State của PomodoroScreen

```
void dispose() {
  _timer.dispose();
  super.dispose();
}
```

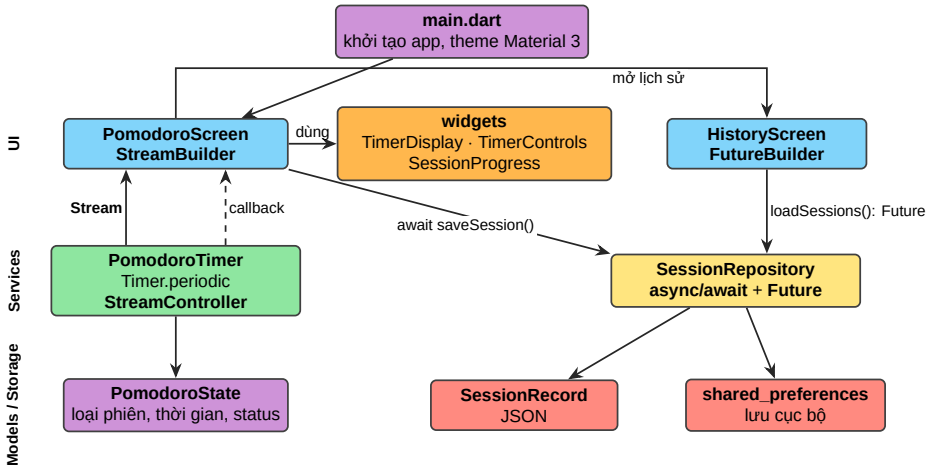
- Flutter gọi **`dispose()`** khi State bị hủy khỏi cây widget.
- **`_timer.dispose()`**: giải phóng bộ đếm mà màn hình sở hữu.
- **`super.dispose()`**: hoàn tất dọn dẹp của lớp State cha.
- StreamBuilder tự hủy subscription; màn hình không tạo subscription thủ công.

2.1 FocusFlow – ứng dụng đồng hồ Pomodoro

- Ứng dụng **Pomodoro Timer** cho **Android**, xây dựng bằng **Flutter/Dart**, **hoạt động offline**.
- **3 chế độ**: Tập trung (mặc định **25'**), Nghỉ ngắn (**5'**), Nghỉ dài (**15'**).
- Điều khiển: **Start, Pause, Resume, Reset**.
- **Tự động chuyển phiên**; **4 phiên tập trung** → nghỉ dài.
- Tuỳ chỉnh thời gian tập trung **1–180 phút**, lưu trên thiết bị.
- **Lịch sử phiên** đã hoàn thành.

Thành phần	Chi tiết
Ngôn ngữ	Dart 3
Framework	Flutter (Material 3)
Bất đồng bộ	`Future` & `Stream` / `StreamController`
Lưu trữ	`shared_preferences`
State	`StreamBuilder`

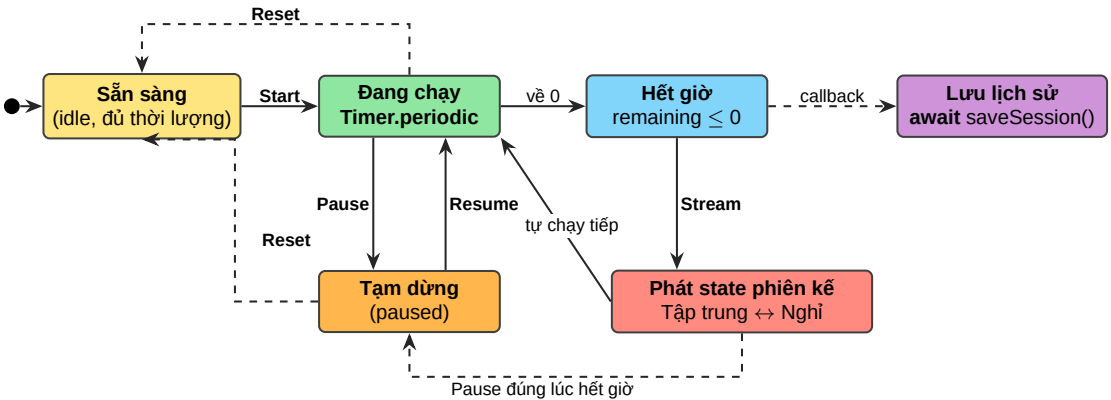
2.1 Kiến trúc trong project



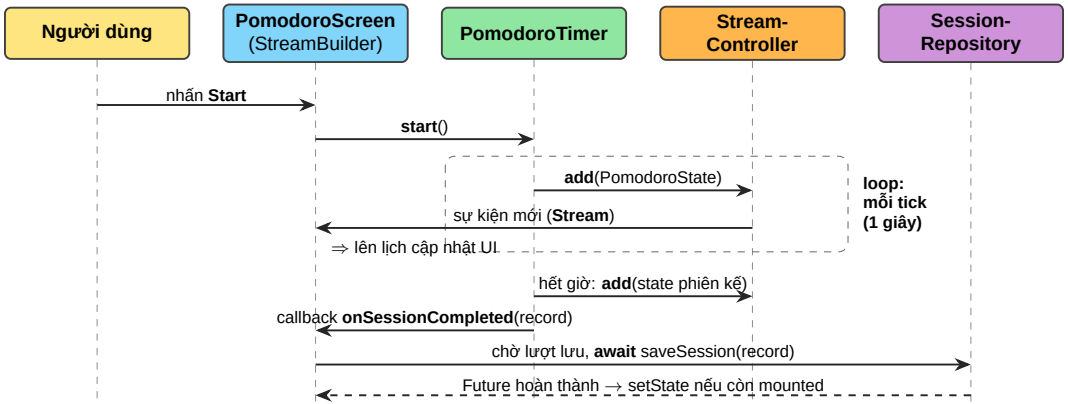
2.2 Áp dụng Future & Stream trong project

Thành phần	Kỹ thuật bất đồng bộ	Vai trò
PomodoroTimer	**`Timer.periodic`** + StreamController (broadcast)	Timer mặc định 1 giây, tính thời gian còn lại và phát state qua Stream
PomodoroScreen	**`StreamBuilder`** + **`Future`**	**Lắng nghe Stream** để cập nhật UI; callback xếp hàng lưu bằng **`_pendingSave.then()`**; Future tải/lưu thời lượng
SessionRepository	**`async/await`** + **`Future`**	Đọc/lưu qua Future; getString và giải mã JSON vẫn là đồng bộ
HistoryScreen	**`FutureBuilder`**	**Một lần đọc**: đang tải / lỗi / có kết quả
Dọn dẹp	**`_ticker?.cancel()`**, **`_stateController.close()`**, **`dispose()`**	Hủy Timer, đóng controller khi PomodoroScreen bị dispose

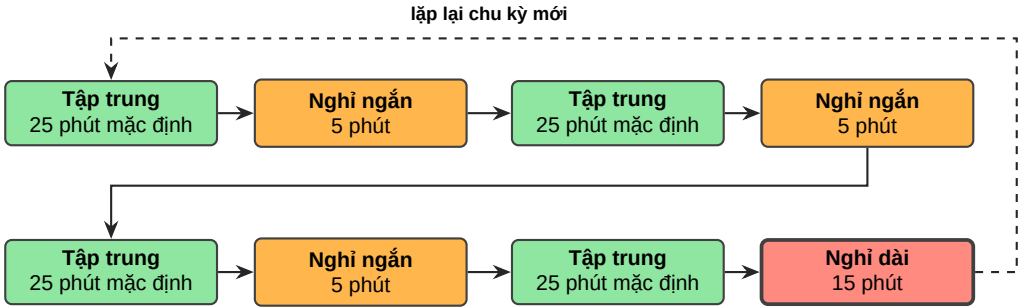
2.2 Sơ đồ trạng thái



2.2 Stream cập nhật UI và Future lưu lịch sử



2.2 Chu kỳ phiên Pomodoro



- Ứng dụng đếm **số phiên tập trung** trong chu kỳ; đủ **4 phiên** thì chuyển sang **nghỉ dài**.
- Hết giờ: phát state phiên kế, callback xếp hàng lưu lịch sử.

2.2 Code bộ đếm và phát Stream

```
void _startTicker() {
  _endTime = _now().add(_state.remaining);
  _ticker?.cancel();
  _ticker = Timer.periodic(tickInterval, (_) => _onTick());
  _emit(_state.copyWith(status: TimerStatus.running));
}
```

```
void _emit(PomodoroState nextState) {
  if (_disposed) return;
  _state = nextState;
  _stateController.add(nextState);
}
```

- **`_endTime`**: mốc kết thúc = hiện tại + thời gian còn lại; dùng để tính lại thời gian mỗi tick.
- **`_ticker?.cancel()`**: hủy Timer cũ nếu có, tránh nhiều bộ đếm chạy cùng lúc.
- **`Timer.periodic`**: gọi **`_onTick()`** theo **`tickInterval`**, mặc định 1 giây.
- **`copyWith(status: running)`**: tạo state mới ở trạng thái đang chạy.
- **`_emit()`**: cập nhật state nội bộ rồi phát qua Stream; bỏ qua nếu đã dispose.

2.2 Code xếp hàng lưu và xử lý lỗi

```
void _saveCompleted(SessionRecord record) {
  _pendingSave = _pendingSave.then((_) => _persistSession(record)
      );
}
```

```
Future<void> _persistSession(SessionRecord record) async {
  try {
    await _repository.saveSession(record);
    if (!mounted) return;
    setState(() {
      if (record.sessionType == SessionType.focus)
     _savedFocusCount++;
      _storageError = null;
    });
  } catch (_) {
    if (mounted) {
      setState(() => _storageError =
            'Không thể lưu phiên vừa hoàn thành.');
    }
  }
}
```

- **`_pendingSave.then()`**: chỉ bắt đầu lần lưu tiếp theo sau lần trước, tránh đọc–sửa–ghi chồng nhau.
- **`await saveSession()`**: đợi lưu xong trước khi cập nhật số phiên trên UI.
- **`mounted`**: nếu màn hình đã bị hủy thì thoát, tránh gọi setState sau dispose.
- **`setState()`**: tăng số phiên nếu là phiên tập trung; xóa thông báo lỗi cũ.
- **`catch`**: bắt lỗi lưu; chỉ hiển thị lỗi khi màn hình còn tồn tại.

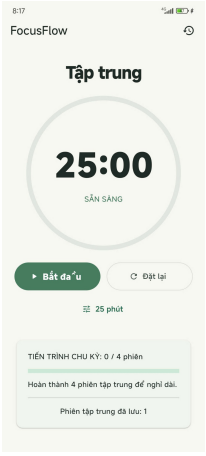
2.2 Code đọc lịch sử bằng FutureBuilder

```
void initState() {
  super.initState();
  _sessions = widget.repository.loadSessions();
}
```

```
body: FutureBuilder<List<SessionRecord>>(
  future: _sessions,
  builder: (context, snapshot) {
    if (snapshot.connectionState != ConnectionState.done) {
      return const Center(child: CircularProgressIndicator());
    }
    if (snapshot.hasError) {
      return const Center(child: Text('Không thể tải lịch
     sử.'));
    }
    final records = (snapshot.data ?? []).reversed.toList();
```

- **`initState()`**: tạo và giữ Future trong **`_sessions`**, tránh đọc lại sau mỗi lần build.
- **`future: _sessions`**: FutureBuilder theo dõi kết quả đọc lịch sử.
- Chưa **`done`**: trả widget vòng tải. Nếu **`hasError`**: trả thông báo lỗi.
- **`snapshot.data ?? []`**: dùng danh sách rỗng nếu chưa có dữ liệu.
- **`reversed.toList()`**: đảo thứ tự để bản ghi thêm sau hiện trước; không sắp xếp theo thời gian.

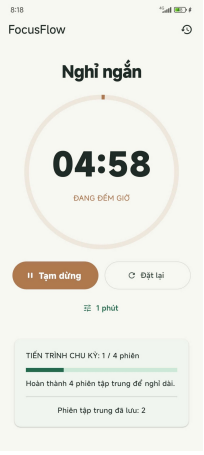
2.3 Demo: màn hình đồng hồ



Trạng thái sẵn sàng

- Phiên tập trung mặc định **25:00**, chưa bắt đầu đếm.
- **Bắt đầu / Đặt lại** điều khiển bộ đếm.
- Nút **25 phút** mở phần chỉnh thời lượng tập trung.
- **0/4** thuộc chu kỳ hiện tại; **đã lưu: 1** thuộc lịch sử trên thiết bị.

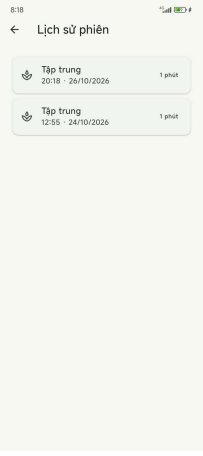
2.3 Demo: chuyển phiên và cập nhật liên tục



Nghỉ ngắn sau phiên tập trung

- Đồng hồ nghỉ còn **04:58**, trạng thái **đang đếm giờ**.
- Chu kỳ đạt **1/4**; có thể bấm **Tạm dừng**.
- **1 phút** là thời lượng tập trung đã đặt để demo nhanh, không phải thời gian nghỉ.

2.3 Demo: lịch sử phiên đã hoàn thành



Đọc lại dữ liệu lưu trên thiết bị

- Ảnh hiển thị **hai phiên tập trung**, mỗi phiên **1 phút**.
- Mỗi bản ghi gồm loại phiên, thời điểm hoàn thành và thời lượng.
- Bản ghi được thêm sau hiển thị trước.

3. Kết luận

- **Future** + **async/await**: chờ **một kết quả hoặc lỗi**, cho phép xử lý sự kiện khác trong lúc chờ.
- **Stream** + **StreamController** + **StreamBuilder**: phát, lắng nghe và cập nhật giao diện theo **luồng dữ liệu liên tục**.
- **Dọn tài nguyên**: hủy Timer và đóng controller; StreamBuilder tự hủy lắng nghe.
- Project **FocusFlow (Pomodoro)** đáp ứng ba yêu cầu: xử lý **Future**, cập nhật bằng **Stream** và **dọn dẹp tài nguyên**.

Xin cảm ơn
thầy và các bạn đã lắng nghe!

Hỏi & Đáp